

# Technische Internetbasierte Systeme

## Hardware-Schnittstellen

Linus Hoppe

Aalen University

April 14, 2026

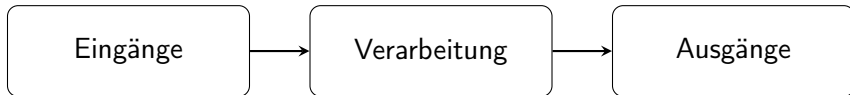




- ① Rückblick
- ② UART / TTL
- ③ RS232 / RS485
- ④ I2C
- ⑤ SPI
- ⑥ OneWire



- ① Rückblick
- ② UART / TTL
- ③ RS232 / RS485
- ④ I2C
- ⑤ SPI
- ⑥ OneWire



- Eingänge: Taster, Sensoren
- Verarbeitung: `loop()`
- Ausgänge: LED, Relais, Motor





## Digital

- HIGH / LOW
- diskrete Zustände

## Analog

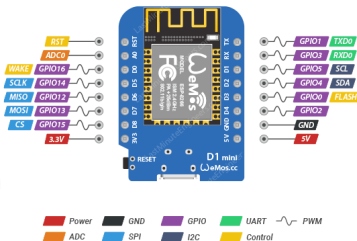
- kontinuierliche Spannung
- Wertebereich statt Zustände

## PWM

- schnelles HIGH / LOW
- Mittelwert wirkt analog

## Einordnung

- Digital → diskret
- Analog → kontinuierlich
- PWM → zeitlich gemittelt





- ① Rückblick
- ② UART / TTL
- ③ RS232 / RS485
- ④ I2C
- ⑤ SPI
- ⑥ OneWire



- asynchrone Übertragung
- kein separates Taktsignal
- Startbit markiert den Beginn
- 7 Datenbits
- 1 Stopbit

## Ablauf

- Leitung in Ruhe: HIGH
- Startbit: LOW
- Datenbits
- Stopbit: HIGH

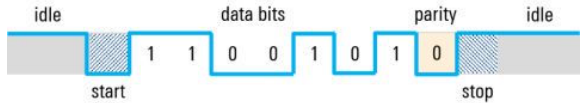




- Paritätsbit optional
- liegt nach den Datenbits
- einfache Fehlerprüfung

**Beispiel: 1010011**

- 4 Einsen
- gerade  $\rightarrow$  Paritätsbit = 0
- ungerade  $\rightarrow$  Paritätsbit = 1

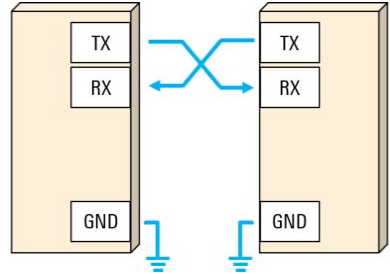


## UART

- asynchrone, bitweise Übertragung
- TX an RX
- gemeinsame Masse (GND)
- Frame: Startbit, Datenbits, optional Parität, Stopbit

## Pegel bei Mikrocontrollern

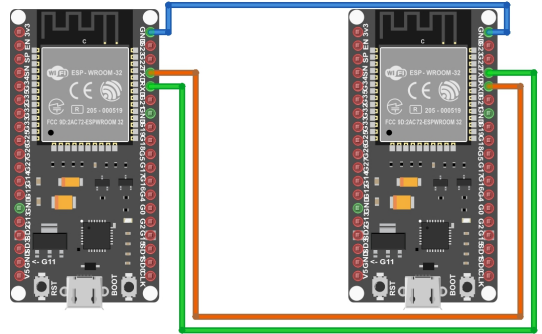
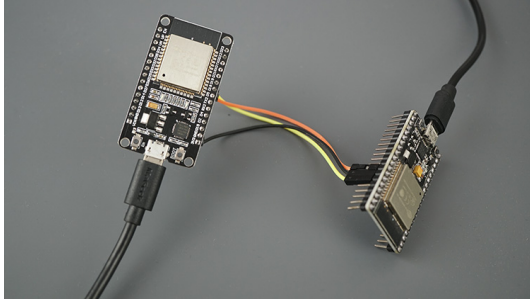
- Arduino / ESP arbeiten meist mit TTL-UART
- ESP typischerweise 3.3 V
- Arduino je nach Board oft 5 V



## Typische Baudraten

- 9600
- 115200

# ESP zu ESP über UART



fritzing

- TX eines ESP an RX des anderen ESP
- RX an TX
- GND gemeinsam verbinden



## ESP A (Sender)

```
void setup() {  
    Serial.begin(115200);  
    Serial1.begin(9600);  
}  
  
void loop() {  
    Serial1.println("Hello from ESP A");  
    delay(1000);  
}
```

- Serial: UART0 (USB / Debugging)
- Serial1: zusätzlicher Hardware-UART
- feste Instanzen im System vorhanden
- direkte Nutzung ohne eigene Definition

## ESP B (Empfänger)

```
void setup() {  
    Serial.begin(115200);  
    Serial1.begin(9600);  
}  
  
void loop() {  
    if (Serial1.available()) {  
        String s = Serial1.readStringUntil('\n');  
        ;  
        Serial.println(s);  
    }  
}
```



## ESP A (Sender)

```
#include <HardwareSerial.h>
HardwareSerial MySerial1(1);

void setup() {
  MySerial1.begin(9600, SERIAL_8N1, 16, 17);
}

void loop() {
  MySerial1.println("Hello");
}
```

- eigener Zugriff auf Hardware-UART
- freie Wahl von RX- und TX-Pins
- sinnvoll bei falscher Board-Vorkonfiguration

## ESP B (Empfänger)

```
#include <HardwareSerial.h>

HardwareSerial MySerial1(1);

void setup() {
  Serial.begin(115200);
  MySerial1.begin(9600, SERIAL_8N1, 16, 17);
}

void loop() {
  if (MySerial1.available())
    Serial.println(MySerial1.readString());
}
```





## ESP A (Sender)

```
#include <SoftwareSerial.h>
SoftwareSerial MySerial(16, 17);

void setup() {
    MySerial.begin(9600);
}

void loop() {
    MySerial.println("Hello");
}
```

- UART wird in Software emuliert
- keine Hardware-Unterstützung
- höhere CPU-Last
- geringere Geschwindigkeit und Stabilität

## ESP B (Empfänger)

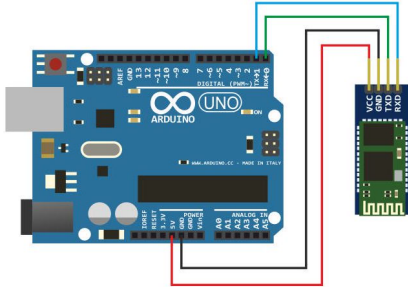
```
#include <SoftwareSerial.h>

SoftwareSerial MySerial(16, 17);

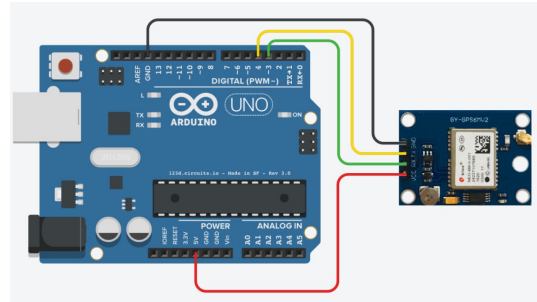
void setup() {
    Serial.begin(115200);
    MySerial.begin(9600);
}

void loop() {
    if (MySerial.available())
        Serial.println(MySerial.readString());
}
```

# Beispiele: UART in der Praxis



HC-05 Bluetooth



NEO-6M Anschluss



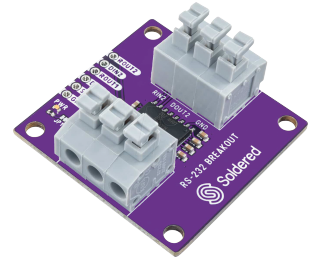
- ① Rückblick
- ② UART / TTL
- ③ RS232 / RS485
- ④ I2C
- ⑤ SPI
- ⑥ OneWire



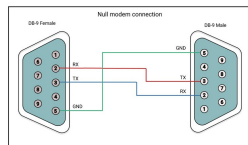
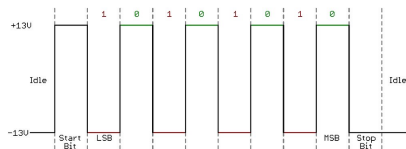
Waage



Steckverbinder



Schnittstellenmodul



## Signalpegel

- logisch 1: negative Spannung
- logisch 0: positive Spannung
- Bereich typischerweise:
  - 1: -3 V bis -15 V
  - 0: +3 V bis +15 V

## Merke

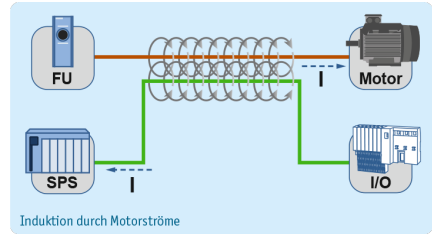
- UART beschreibt die Datenstruktur
- RS232 beschreibt die elektrischen Pegel
- Punkt-zu-Punkt-Verbindungen

## Grenzen von RS232

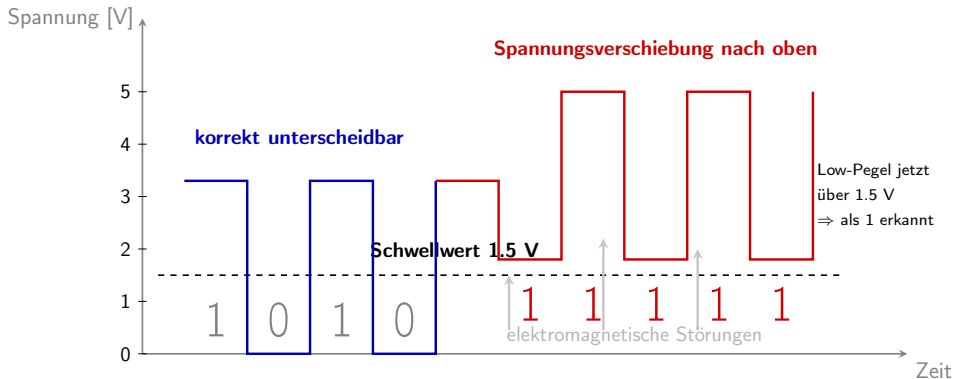
- Punkt-zu-Punkt
- begrenzte Reichweite
- störanfällig bei längeren Leitungen
- Signal gegen GND (Single-Ended)

## Störquellen

- elektromagnetische Felder
- Motoren und Umrichter
- industrielle Umgebung



# Störanfälligkeit bei Single-Ended-Übertragung



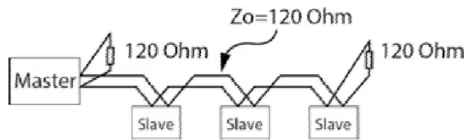
Didaktisch vereinfachte Darstellung: Äußere Störungen können das Signal insgesamt nach oben verschieben. Dadurch liegen die Low-Pegel nicht mehr unter dem Schwellwert, sondern darüber und werden deshalb fälschlich als High erkannt.

## Grundprinzip

- differentielle Übertragung (A / B)
- Information:  $V_{diff} = V_A - V_B$
- absolute Spannung ist nicht entscheidend

## Technische Daten

- Reichweite: bis zu 1200 m
- Datenrate:
  - bis zu 10 Mbit/s (kurze Strecken)
  - geringer bei langen Leitungen



## Topologie

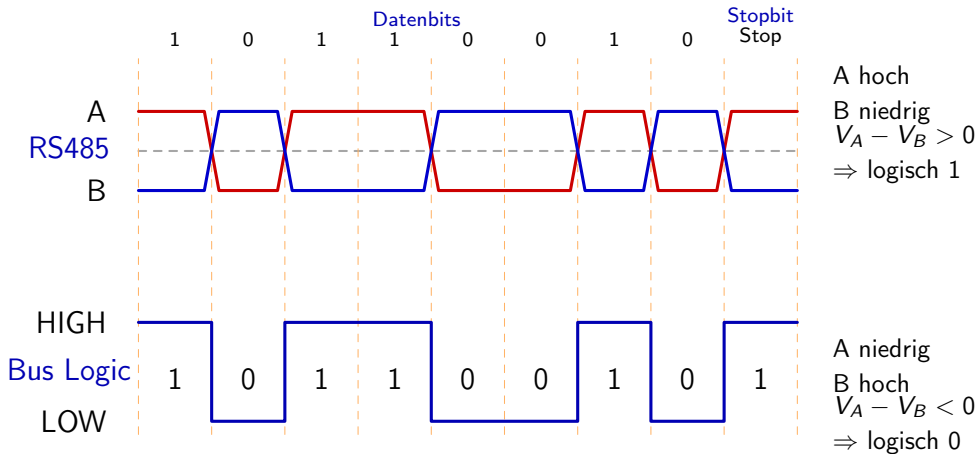
- Bus-System (mehrere Teilnehmer)
- typ. bis zu 32 Geräte

## Eigenschaften

- robust gegenüber Störungen
- gleiche Störung auf beiden Leitungen
- Differenz bleibt erhalten



# RS485 – Differenzbildung über mehrere Bits



Beispielhafte Darstellung über 8 Datenbits und ein Stopbit: Entscheidend ist die Differenz zwischen A und B, nicht die absolute Spannung einer einzelnen Leitung.

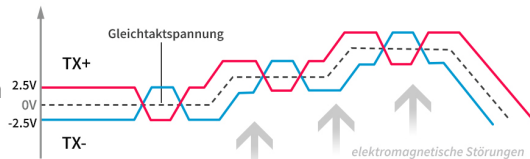


## Ausgangspunkt (RS232)

- Signal wird gegen Masse gemessen
- Störungen verschieben das gesamte Signal
- Schwellwert kann überschritten werden

## RS485 Ansatz

- zwei Leitungen: A und B
- Auswertung über Differenz:
  - $V_{diff} = V_A - V_B$



## Merke

Bei RS485 wirkt eine Störung auf beide Leitungen nahezu gleich. Da nur die Differenz ausgewertet wird, bleibt das Signal eindeutig.



**H<sub>2</sub>S Sensor**

- misst giftiges Gas (Schwefelwasserstoff)
- Einsatz: Industrie, Kläranlagen
- robuste Messung auch bei Störungen

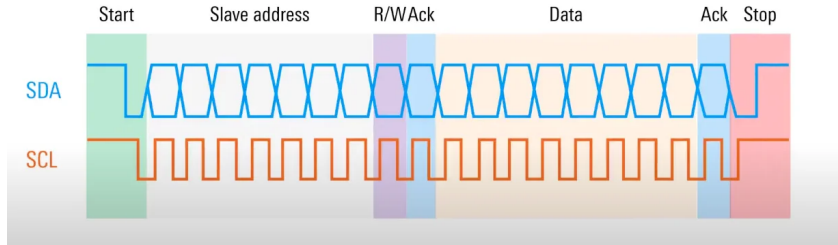


**Neigungssensor**

- misst Winkel / Lage
- Einsatz: Maschinen, Kräne
- hohe Genauigkeit und robust



- ① Rückblick
- ② UART / TTL
- ③ RS232 / RS485
- ④ I2C
- ⑤ SPI
- ⑥ OneWire



## Definition

- I2C = Inter-Integrated Circuit
- synchrones Busprotokoll

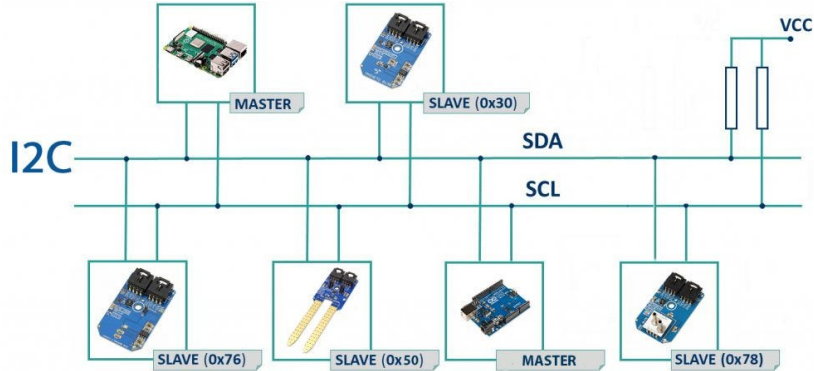
## Leitungen

- SDA (Daten)
- SCL (Takt)

## Eigenschaft

- mehrere Teilnehmer auf einem Bus

# I2C – Adressierung



- Master spricht gezielt ein Gerät über Adresse an
- Adresse ist im Datenblatt angegeben
- typischerweise 7-Bit Adressen

## Problem

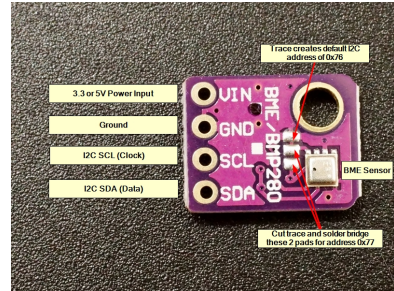
- jede Adresse darf nur einmal im Bus vorkommen
- gleiche Sensoren → gleiche Adresse
- Kommunikation nicht eindeutig möglich

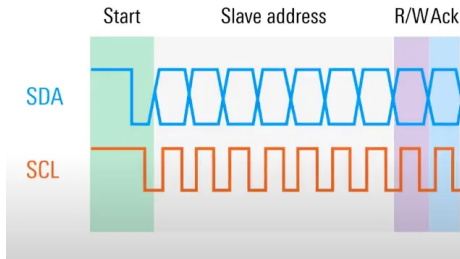
## Ursache

- feste Adressen im Sensor
- mehrere identische Geräte im System

## Lösung

- alternative Adressen vorhanden
- Auswahl über Pins oder Lötbrücken





## Start

- Master erzeugt Startbedingung
- alle Geräte erkennen Bus aktiv

## Adressierung

- 7-Bit Adresse + R/W Bit
- nur passendes Gerät reagiert

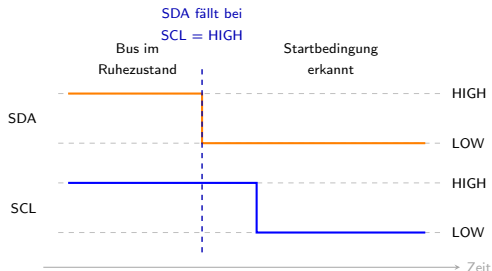
## Richtung

- Write (0): Master → Slave
- Read (1): Slave → Master

## ACK

- adressiertes Gerät bestätigt
- andere bleiben passiv





## Definition

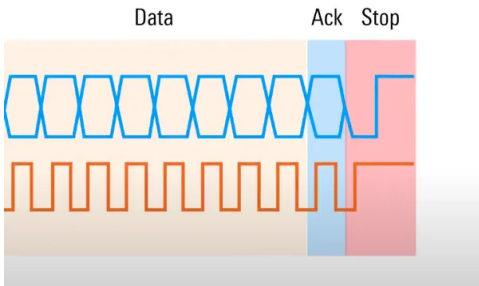
- START: SDA wechselt auf LOW
- dabei bleibt SCL auf HIGH

## Bedeutung

- der Master beginnt die Kommunikation
- alle Geräte erkennen: Bus wird aktiv

## Wichtig

- Daten ändern sich sonst nur bei SCL = LOW
- genau deshalb ist der START eindeutig erkennbar



## Datenübertragung

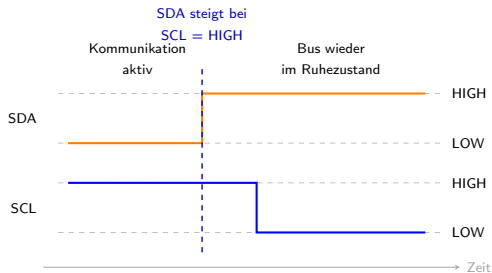
- Übertragung in Bytes (8 Bit)
- nach jedem Byte folgt ACK

## Ablauf

- Master sendet oder empfängt Daten
- Takt wird durch Master vorgegeben

## Ende

- Master erzeugt Stopbedingung
- Bus wird wieder freigegeben



## Definition

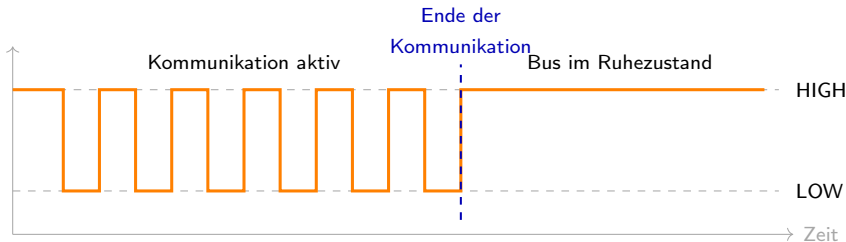
- STOP: SDA wechselt auf HIGH
- dabei bleibt SCL auf HIGH

## Bedeutung

- Kommunikation wird beendet
- Bus geht in den Ruhezustand über

## Wichtig

- eindeutig erkennbar durch Wechsel bei SCL = HIGH

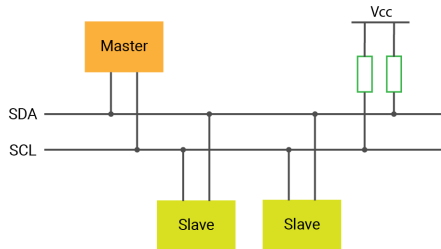


## Ruhezustand

- SDA und SCL liegen auf HIGH
- keine Kommunikation aktiv

## Frage

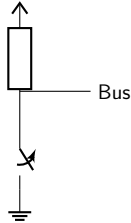
Welche Schaltung erzeugt diesen HIGH-Zustand ohne aktives Treiben?



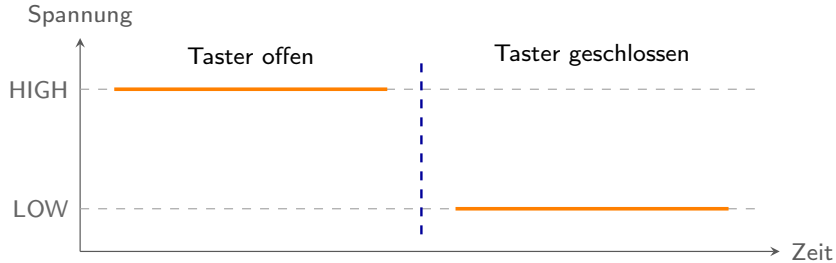
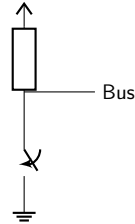
# I2C – Pull-Up-Schaltung



Taster offen



Taster geschlossen





## Übertragung auf I2C

- SDA und SCL liegen im Ruhezustand auf HIGH
- kein Teilnehmer treibt aktiv auf HIGH
- ein Teilnehmer zieht die Leitung nur bei Bedarf auf LOW

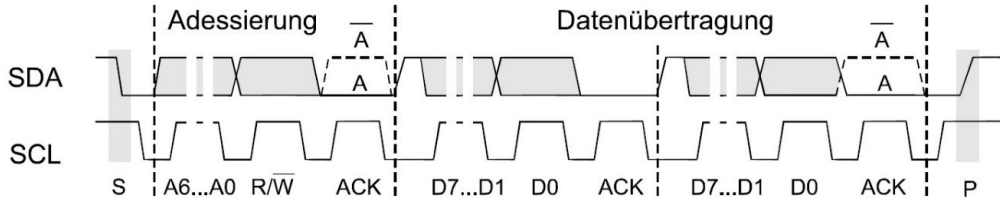
## Folge

- mehrere Teilnehmer können dieselbe Leitung nutzen
- entscheidend ist nur, wer die Leitung auf LOW zieht

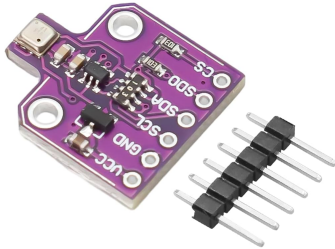
## Merke

Das passt genau zum I2C-Bus: HIGH im Ruhezustand, LOW durch aktives Ziehen.

# I2C – Gesamter Ablauf



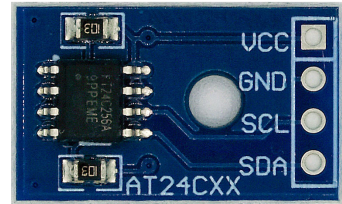
# I2C – Beispiele



Sensor



Display



EEPROM



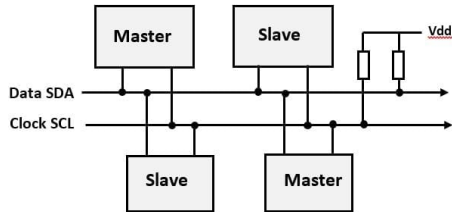


## Prinzip

- 2 Leitungen: SDA und SCL
- Bus mit mehreren Teilnehmern
- Pull-Ups erzeugen den HIGH-Zustand

## Ablauf

- Start
- Adresse + R/W-Bit
- Daten in Bytes
- nach jedem Byte: ACK
- Stop





## Definition

- I3C = Improved Inter-Integrated Circuit
- Nachfolger von I2C
- ebenfalls Bus mit zwei Leitungen

## Ziele

- höhere Datenrate
- geringerer Energiebedarf
- weniger zusätzliche Signalleitungen



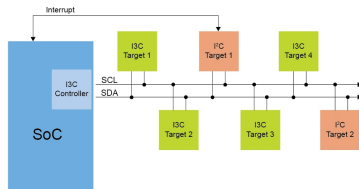
## Wichtig

- viele I2C-Geräte können weiterhin eingebunden werden
- moderne Lösung für Sensor- und Peripherieanbindung



## Gegenüber I2C

- höhere Datenrate
- Push-Pull möglich, nicht nur Open-Drain
- dynamische Adressvergabe
- Interrupts direkt über den Bus



## Vorteile

- schneller
- effizienter
- weniger Zusatzleitungen nötig

## Einordnung

- I2C ist weit verbreitet
- I3C ist leistungsfähiger
- in der Praxis bisher noch seltener



## Bosch BMI263

- IMU (Beschleunigung + Gyroskop)
- misst Bewegung und Lage
- Einsatz: Smartphones, Wearables



## ST VL53L9CA

- Time-of-Flight Distanzsensor
- misst Abstand und erstellt Tiefenbild
- Einsatz: Kamera, Gestenerkennung



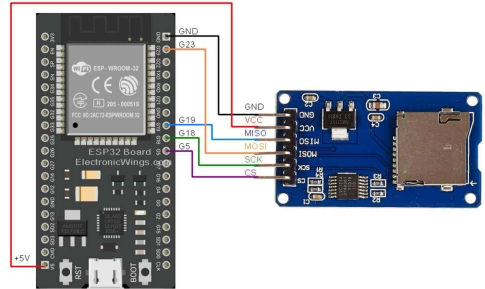
- ① Rückblick
- ② UART / TTL
- ③ RS232 / RS485
- ④ I2C
- ⑤ SPI**
- ⑥ OneWire

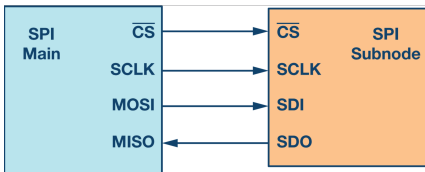
## Eigenschaften

- sehr hohe Datenraten (bis 50 MHz)
- synchrone Übertragung
- Vollduplex (Senden und Empfangen)

## Topologie

- mehrere Geräte möglich
- jedes Gerät benötigt eigene CS-Leitung





## CS / SS

- aktiviert genau ein Gerät
- nur das ausgewählte Gerät reagiert

## SCLK / SCK

- Taktsignal vom Master
- synchronisiert die Datenübertragung

## MOSI

- Daten vom Master zum Slave

## MISO

- Daten vom Slave zum Master



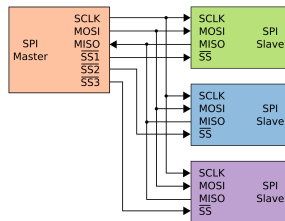
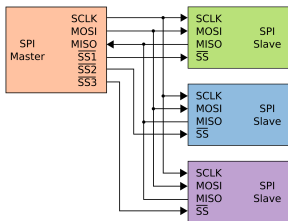
## Funktion

- CS wird vom Master gesteuert
- LOW aktiviert genau ein Gerät
- HIGH deaktiviert das Gerät

## Folge

- nur der ausgewählte Slave sendet/empfängt Daten
- alle anderen bleiben inaktiv





## Stern

- jedes Gerät hat eigene CS-Leitung
- direkte Auswahl durch den Master

## Kaskade

- Geräte hintereinander geschaltet
- Daten laufen durch alle Geräte

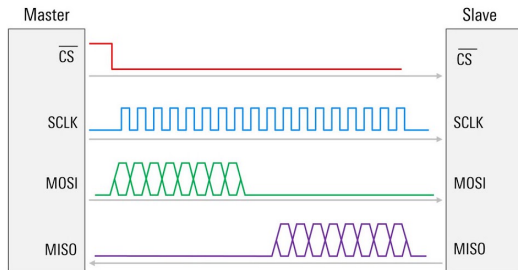


## Prinzip

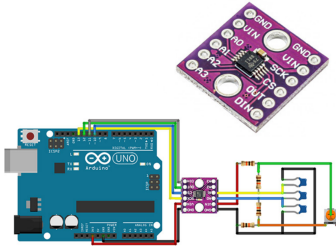
- getrennte Leitungen für Senden und Empfangen
- MOSI und MISO gleichzeitig aktiv

## Vorteile

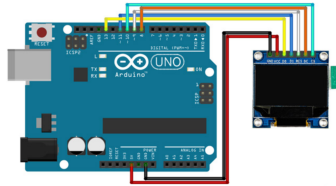
- Vollduplex: gleichzeitiges Senden und Empfangen
- keine Umschaltung der Richtung nötig
- hohe Datenrate möglich



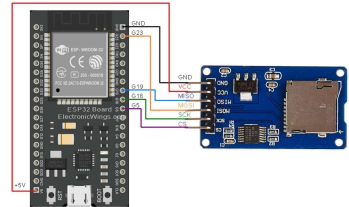
# SPI – Beispiele



ADC



Display (LCD)



SD-Karte



- ① Rückblick
- ② UART / TTL
- ③ RS232 / RS485
- ④ I2C
- ⑤ SPI
- ⑥ OneWire

## Eigenschaften

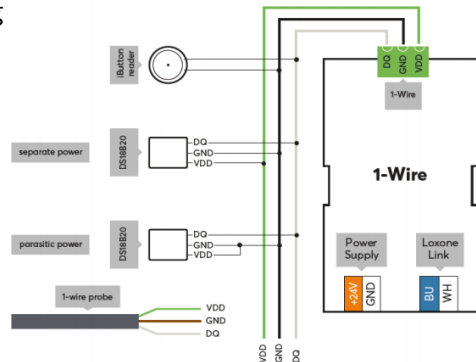
- Kommunikation über nur eine Datenleitung
- zusätzlich gemeinsame Masse (GND)
- jedes Gerät hat eindeutige ID

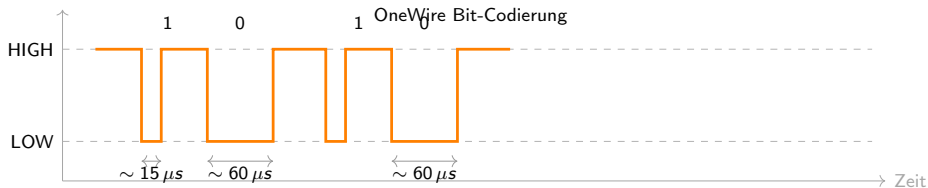
## Nachteile

- geringe Datenrate
- empfindlich bei längeren Leitungen

## Prinzip

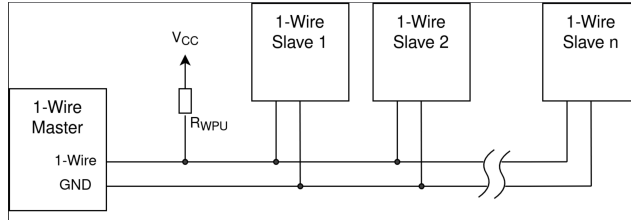
- Master steuert den Bus
- Kommunikation über definierte Zeitfenster





## Prinzip

- kurzer LOW-Puls → 1
- langer LOW-Puls → 0



## Grundprinzip

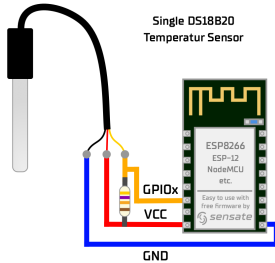
- nur eine Datenleitung + GND
- Master-Slave-System
- mehrere Geräte auf einem Bus möglich
- jedes Gerät besitzt eine eindeutige ID

## Vorteile

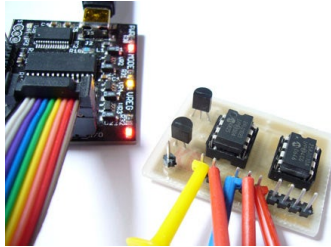
- sehr wenig Verdrahtung
- einfache Integration
- eindeutige Adressierung

## Nachteile

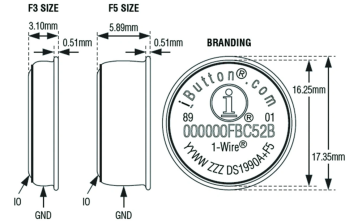
- geringe Datenrate
- timingkritisch



Temperatursensor



Speicher (EEPROM)



iButton / ID





| Protokoll  | Datenrate               | Reichweite  | Teilnehmer | Topologie       |
|------------|-------------------------|-------------|------------|-----------------|
| OneWire    | ~16 kbit/s              | ~10–100 m   | mehrere    | Bus             |
| I2C        | 100 kbit/s – 3.4 Mbit/s | ~1 m        | viele      | Bus             |
| UART (TTL) | bis ~5 Mbit/s           | ~1–10 m     | 2          | Punkt-zu-Punkt  |
| RS232      | ~20 kbit/s              | ~15 m       | 2          | Punkt-zu-Punkt  |
| RS485      | bis ~10 Mbit/s          | bis ~1200 m | viele      | Bus             |
| SPI        | bis ~50 MHz             | ~<1 m       | viele      | Stern / Kaskade |

## Einordnung

- langsam → OneWire, I2C
- mittel → UART, RS232
- robust / weit → RS485
- schnell → SPI